TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

JBI010 - Assignment (2026-2027)

Spotify Music Recommendation and Playlist Generation

Contributor 1: /Replace with your full name/

TU/e ID number 1: /Replace with your TU/e ID number/

Contributor 2: /Replace with your full name/

TU/e ID number 2: /Replace with your TU/e ID number/

Date: /Replace with the date/

Introduction

Every day, millions of people use music streaming platforms to discover songs, follow artists, create playlists, and listen to music that matches their mood. Behind these platforms are large datasets that describe each track through measurable properties, such as genre, duration, and audio features. This data can help us answer interesting questions such as: How can similar tracks be recommended? How can audio features help explain recommendation patterns? What makes a playlist coherent?

In this assignment, you will build an object-oriented Python program to load, validate, and analyze a real-world music dataset. You will also design classes and methods to represent the data and support different forms of analysis. Because real-world data are rarely perfect, you must write robust code that handles incomplete or invalid input gracefully. Beyond producing correct output, the goal is to deliver clean, maintainable code supported by meaningful **`pytests`** and to reflect thoughtfully on your design process.

Additionally, the assignment includes a bonus task that involves building an interactive dashboard using **Dash**, a Python framework for creating data applications. Although Dash is not formally covered in the course, independent exploration using online resources and tutorials is expected. This task reuses the classes and methods developed in previous tasks to visualize recommendations and generated playlists.

Grading

The assignment contains **6 Tasks**, including **1 bonus Task**. Each task contributes differently to the final grade. To help ensure correctness, students are expected to write their own **`pytests`** for the code they implement. These tests should check whether the required functionality works as intended across different situations. Submitted solutions will also be checked using an internal test suite maintained by the course staff. These tests are hidden from the students. Passing both the student-written and staff-maintained tests is important but does not automatically guarantee full marks. The final grade also depends on code quality, structure, and adherence to the assignment instructions.

All required functions and methods must include **type hints** and a **clear, meaningful docstring** that explains their purpose. You must also follow the **Python Coding Standard** provided on Canvas. Missing docstrings, unclear type hints, poor structure, or unnecessary complexity may lead to point deductions.

The final assessment will consider the correctness of the program, the quality of the code, the relevance and completeness of the student-written tests, the results of the internal staff tests, and the quality of the open-ended tasks involving analysis and reflection.

Use of AI Statement

You must include a clear and honest statement in your reflection report describing whether and how you used AI during the assignment. Generating partial or complete solutions with **AI is not allowed**. The final implementation must be written, tested, and understood by you. However, AI tools can be used to clarify concepts or ask for general explanations. The following AI usage examples help to clarify the distinction between what is allowed and not allowed:

- **Allowed:** Using AI to understand Python concepts, error messages, or documentation.
- **Not allowed:** Using AI to generate code.

Your statement should specify which AI tool you used, what you used it for, and how it influenced your final work. This statement is **mandatory,** and failure to provide it may result in a deduction of points. If AI use is suspected, an oral examination based on the assignment will be conducted, and the grade obtained in the oral examination will replace the assignment grade.

Tests

The tests folder contains test files with a small number of predefined **pytests** for Tasks 2, 3, and 4. These **pytests** are provided to guide your testing process and they are not exhaustive. You are expected to extend them into a comprehensive test suite that checks correctness of each required method under different conditions. All tests must use **pytest**. Do not use unittest or another testing framework.

In addition to your own tests, the final submission will also be evaluated using an internal hidden test suite maintained by the course staff. Because these tests are not visible to you, you should not design your solution around the visible tests alone. Instead, follow the assignment specification carefully and handle edge cases in a robust way.

Tasks

Each task **must be** implemented in the **file indicated** for that task. **Do not rename, move, or delete the provided files**. The file names, folder structure, class names, and method names are used during

automated testing and evaluation, and changing them may result in point deductions. Implement each method as specified, using the stated parameters and return types, so that your solution remains compatible with the automated grading test suite.

Dataset

This assignment uses a cleaned and modified version of the publicly available Spotify Tracks Dataset from Kaggle. The original dataset covers tracks from a wide range of genres and includes track-level information such as artist names, track titles, popularity scores, and Spotify-provided audio features.

To better match the learning objectives of this course, the original dataset was modified before being provided in this assignment. Some columns were removed, some values were cleaned, and the original Spotify genres were grouped into broader categories. In addition, entries containing explicit content, offensive or inappropriate text, duplicate records, or other unsuitable material were removed to ensure that the dataset is appropriate for educational use.

The final version used in this assignment includes the following fields:

- **track_id:** A unique Spotify identifier for the track (for example, `4yNk9iz9WVJikRFle3XEvn`).
- **artists:** The name of the artist or artists who performed the track. This field can have multiple artists separated by a semicolon (for example, `Calvin Harris;Dua Lipa;Oliver Heldens`).
- **track_name:** The name of the track (for example, `As It Was`).
- **genre:** The broad categorical genre assigned to the track (for example, `Folk`).
- **mood:** A categorical mood label for the track (one of `Sad, Neutral, or Happy`).
- **duration_category:** A categorical duration label for the track based on the track's length (one of `Short, Medium, or Long`).
- **danceability:** A floating-point value between 0.0 and 1.0 describing how suitable a track is for dancing. Higher values correspond to tracks that are more danceable, while lower values correspond to tracks that are less suitable for dancing (for example, `0.515`).
- **energy:** A floating-point value between 0.0 and 1.0 representing the intensity of the track. Higher values correspond to more energetic and intense tracks, while lower values correspond to calmer or softer tracks (for example, `0.593`).
- **valence:** A floating-point value between 0.0 and 1.0 describing the musical positivity conveyed by the track. Higher values correspond to a more positive or upbeat character, while lower values correspond to a more negative one (for example, `0.153`).
- **tempo:** The tempo of the track in beats per minute. It must be a floating-point value that is greater than 0. Higher values of tempo correspond to faster tracks, while lower values correspond to slower tracks (for example, `94.467`).

- **loudness:** The overall loudness of the track in decibels (dB), represented as a floating-point value that may be negative or positive. Higher values of loudness correspond to louder tracks, while lower values correspond to quieter tracks (for example, `-4.801 or 0.92`).
- **acousticness:** A value between 0.0 and 1.0 estimating whether the track is acoustic. Higher values correspond to tracks that are more likely to be acoustic, while lower values correspond to tracks that are less likely to be acoustic (for example, `0.653`).
- **duration_ms:** The duration of the track in milliseconds. Higher values correspond to longer tracks, while lower values correspond to shorter tracks. This value must be greater than 0 (for example, `209259`).
- **popularity:** An integer score from 0 to 100, where higher values indicate greater popularity on Spotify. A value of 0 is valid and indicates very low popularity (for example, `66`).

`Danceability`, `energy`, `valence`, `tempo`, `loudness`, and `acousticness` are floating-point values with up to three decimal places. Store these values without rounding or truncating them.

Task 1: Create Issues on GitLab (5 Points)

Version control systems such as Git are essential for tracking changes and enabling effective collaboration, particularly in team-based software projects. Since this assignment is designed for teamwork, the TU/e GitLab **must** also be used to organize the work between team members.

For this task, you will use the **TU/e GitLab issue tracking system** to plan, organize, and document your work. Team members should use this service to coordinate collaboration and development. For each task, you are required to do the following:

a) Create Issues

- Create at least one **GitLab issue per required assignment task**, and one for the bonus task if it is attempted.
- You may also create smaller subtasks to support clearer planning and an even work distribution.

b) Use Clear Commit Messages

- All team members must commit and push their work regularly and divide the workload evenly.
- Write meaningful commit messages (for example, `Add Track class constructor` or `Implement CSV validation`).
- You may optionally use Conventional Commits to structure your commit messages.

c) Close Issues When Completed

- Close each issue once the corresponding work has been completed.
- On GitLab, an issue can be closed automatically by including a predefined keyword such as "close", followed by the issue number, in a commit message or merge request description

(for example, `close #10`). The issue closes automatically once the commit reaches the default branch or the merge request is merged. It may also be closed manually via the web interface.

d) Collaborate Effectively

- Pull your team member's changes regularly before continuing your own work, and push your own changes often so that your partner can follow your progress.
- If a merge conflict occurs, resolve it locally before pushing again. Both team members are expected to contribute actively and to keep the repository organized.

Task 2: Define the Core Data Model (35 Points)

This task forms the foundation of the assignment. You will design two core classes: one to represent an individual track, and another to represent the entire music catalogue. Your goal is to apply object-oriented programming (OOP) principles to create structures that are clean, reusable, and extensible.

Note: The assignment-wide AI policy described in the **Use of AI Statement** section applies to this task, as it does to all other tasks in this assignment. You must design and implement the classes on your own.

Attribute requirements: You are responsible for identifying which attributes each class needs to support its methods. Where an attribute corresponds to a dataset field, it must be named exactly as that field is listed on pages 3–4, because the automated tests read the attributes by name.

Method requirements: Each method must be implemented exactly as described, using the specified parameters and return types, to ensure compatibility with the automated grading test cases.

Library usage: You must only use the Python libraries imported in the provided project skeleton. Do not import or use any other libraries unless explicitly permitted in the instructions.

Part A: Implement the Track Class (15 Points)

The `Track` class represents a single track from the dataset and stores the dataset fields associated with that track. Implement the methods below in the provided track.py file.

Required methods:

- **`__init__`**

  Initializes all attributes corresponding to the dataset fields for a single track object.

  Note: The constructor parameters **must** follow the same order as the fields listed in the **Dataset** section above on pages 3-4 of this assignment and each attribute must be stored under that same field name. This fixed order and naming is required for compatibility with the automated grading tests.

  Parameters: Relevant attributes matching the dataset fields.

  Return: None.

- **__str__**

  Returns a string representation of the track in the format shown below. The output must match this layout exactly, including field names, order, punctuation, and spacing, because the automated tests compare it character for character.

```
Format: TrackID: <track_id>, Title: <track_name>, Artist: <artists>, Genre: <genre>

Example 1: TrackID: 4yNk9iz9WVJikRFle3XEvn, Title: golden hour, Artist: JVKE, Genre: Pop

Example 2: TrackID: 3gE4eQH3K83Sght0ZLvuBK, Title: MIA, Artist: Bad Bunny;Drake, Genre: Latin
```

  Parameters: None.

  Return: Text.

- **artist_names**

  Returns the individual artists who performed the track as a list of strings. Because the artists field may contain several collaborating artists separated by a semicolon, this method must split that field on the semicolon and strip any surrounding whitespace. A track with a single artist returns a one-element list.

  Parameters: None.

  Return: List of individual artist names.

  Example: For a track whose artists field is "`Calvin Harris;Dua Lipa;Oliver Heldens`", the method returns ["`Calvin Harris`", "`Dua Lipa`", "`Oliver Heldens`"]. For "`JVKE`", it returns ["`JVKE`"].

- **audio_features**

  Returns the track's six numeric audio descriptors as a dictionary mapping each feature name to its value: `danceability`, `energy`, `valence`, `tempo`, `loudness`, and `acousticness`. This dictionary is reused in Task 3 to measure similarity between tracks, and in bonus Task 5 to drive the dashboard visualizations.

  Parameters: None.

  Return: Dictionary mapping each audio-feature name to its value.

  Example: For a track with `danceability` 0.515, `energy` 0.593, `valence` 0.153, `tempo` 94.467, `loudness` -4.801, and `acousticness` 0.653, the method returns:

```
{"danceability": 0.515, "energy": 0.593, "valence": 0.153, "tempo": 94.467, "loudness": -4.801, "acousticness": 0.653}
```

Testing: A sample test file, test_track.py, is provided as a starting point in the tests folder. You must extend it into a comprehensive test suite without changing the location or name of the file. Your **pytests** should check whether each required method behaves correctly for normal cases, edge cases, and invalid input where relevant.

Part B: Implement the MusicLibrary Class (20 Points)

The `MusicLibrary` class stores the complete collection of tracks and is responsible for loading them in a structured format. It reads the CSV file, located at data/spotify_tracks.csv, validates and cleans each row, and creates the corresponding `Track` objects. Implement the class in the provided music_library.py.

Required methods:

- **__init__**

  Initializes a new `MusicLibrary` object with an empty list (`self.tracks`) designed to store multiple `Track` objects.

  Parameters: None.

  Return: None.

- **load_from_csv**

  Reads and processes each row of the CSV file to create a `Track` object. It performs typecasting by converting `duration_ms` and `popularity` to integers, converting `danceability`, `energy`, `valence`, `tempo`, `loudness`, and `acousticness` to floating-point values, and storing the remaining fields as strings. A row whose value cannot be converted to the required type is treated as invalid and is skipped. It also validates each row according to the **Validation rules** stated later in this section.

  Parameters: The path to the CSV file.

  Return: None.

- **__len__**

  Returns the number of `Track` objects successfully loaded into the `MusicLibrary`.

  Parameters: None.

  Return: Integer count of stored tracks.

- **get_track**

  Returns the `Track` object whose `track_id` matches the given identifier, or None if no such track exists in the `MusicLibrary`.

  Parameters: The `track_id` to look up.

  Return: `Track` object, or None if the identifier is not present.

Validation rules: The following validation rules apply when loading the dataset in `load_from_csv()`:

- Categorical values may be capitalized inconsistently in the CSV file. Normalize each value such that every word starts with an uppercase letter and its remaining letters are lowercase, treating a hyphen as a word separator (for example, `pop, poP, pOP, and POP all become Pop, and hip-hop becomes Hip-Hop`).
- All categorical fields must be validated against their fixed sets:

  - `genre` ∈ {Pop, Rock, Metal, Hip-Hop, Electronic, Jazz, Soul, Folk, Country, Latin, World, Classical}.
  - `mood` ∈ {Sad, Neutral, Happy}.
  - `duration_category` ∈ {Short, Medium, Long}.

  Any row containing empty or misspelled values in the categorical fields listed above must be excluded from the library.

- The numeric fields and their ranges must be validated:
  - `popularity` is an integer in [0, 100].
  - `danceability`, `energy`, `valence`, and `acousticness` are floats in [0.0, 1.0].
  - `tempo` is a float greater than 0, and `duration_ms` is an integer greater than 0.
  - `loudness` is a float that may be negative or positive.

  Any row containing an empty value, a value that cannot be converted to the required type, or a value outside its stated range in the numeric fields above must be excluded from the library.

- Column headers must be normalized before processing: convert all letters to lowercase, and convert multi-word headers separated by a space or a hyphen to snake_case (for example, `Track Name` and `Track-Name` both become `track_name`).
- If a `track_id` appears more than once in the CSV file, all rows with that `track_id` must be excluded from the library.

Implementation tip: Dataset validation must be performed in `load_from_csv()` or in helper methods called by it. Using small helper methods is recommended to keep the logic clear and reusable. The `__init__` method of the `Track` class should only store the values it receives and must not perform dataset validation.

Robustness: The file path given to `load_from_csv()` always exists and can be opened, and all column headers are valid after normalization. You therefore do not need to handle missing or unreadable files, or headers that match no dataset field. Headers may still be capitalized differently or separated by a space or a hyphen instead of an underscore, so they must be normalized before use. The contents of the file may also be imperfect, and your method must never crash because of them. The CSV files used for testing may contain intentionally corrupted rows, such as an invalid value like 'Maybe' for a categorical field, 150 for `popularity`, or a row with fewer columns than the header. In addition, some track names legitimately contain commas or quotation marks, so you must read the file with a proper CSV parser rather than splitting lines by hand. Therefore, your method must correct the capitalization of headers and categorical values, and exclude rows with invalid or empty values, as well as all rows with a duplicate `track_id`.

Testing: A sample test file, test_music_library.py, is provided as a starting point in the tests folder. You must extend it into a comprehensive test suite without changing the location or name of the file. Your **pytests** should check whether each required method behaves correctly for normal cases, edge cases, and invalid input where relevant.

Task 3: Track Similarity and Recommendation (30 Points)

In this task, you will create a new class named `MusicAnalyzer` that analyzes the tracks loaded in Task 2. The class measures how similar two tracks are by comparing their audio features, and then uses this to recommend the tracks that sound most like a chosen one. This reflects a simple version of a music recommendation system, where one track is used as a starting point to find other tracks with similar audio characteristics. You must implement the class in the provided music_analyzer.py. This task builds directly on Task 2. Therefore, its methods should reuse previously implemented functionality where appropriate.

Library usage: You may only use the Python libraries imported in the provided file (for example, `math library for calculating the square root`). Do not use any other external library.

Part A: Feature Normalization (13 Points)

The six audio features are measured on different scales. For example, `valence` is between 0 and 1, while `tempo` is measured in beats per minute and `loudness` is measured in decibels. If these values are compared directly, features with larger numerical ranges may dominate the similarity calculation. To make the comparison fair, each feature must first be rescaled to a common range. In this task, **min-max normalization** is used to convert each feature value to a number between 0 and 1, based on the smallest and largest values of that feature. The exact formula for min-max normalization is provided in the Appendix.

Required methods:

- **__init__**

  Stores the given `MusicLibrary` object in an attribute named `library`, so that the other methods can access the tracks it contains.

  Parameters: `MusicLibrary` object containing the tracks to analyze.

  Return: None.

- **feature_ranges**

  Returns the minimum and maximum value of each of the six audio features across all tracks in the library. These ranges are used to normalize the feature values consistently. If the library contains no tracks, the method returns an empty dictionary.

  Parameters: None.

  Return: Dictionary mapping each audio-feature name to a (minimum, maximum) tuple, or an empty dictionary if the library contains no tracks.

- **normalized_features**

  Returns the min-max-normalized audio features of the track with the given `track_id`, as a dictionary mapping each feature name to a value in [0.0, 1.0]. The method must reuse `get_track`, `audio_features`, and `feature_ranges`. Use the min-max normalization formula provided in the Appendix. If a feature's minimum and maximum are equal, its normalized value must be defined as **0.0** so that division by zero is never attempted. This

includes the case where the library contains only one track. If the given `track_id` does not exist, the method must raise a `ValueError`. The exact wording of the `ValueError` message is not assessed.

Parameters: `track_id`.

Return: Dictionary mapping each audio-feature name to its normalized value.

Part B: Distance and Recommendation (17 Points)

In Part A, you rescaled the audio features so that they can be compared fairly. In this part, you will measure the distance between two tracks and use that distance to recommend similar music. To recommend tracks similar to a selected track, compare it with every other track in the `MusicLibrary` and return the closest matches. The number of tracks to return is denoted by **k**. For example, **k = 5** returns the five most similar tracks.

Required methods:

- **distance**

  Returns the **Euclidean distance** between two tracks based on their normalized audio features. The method receives two **`track_id`** values as input, retrieves the corresponding tracks, and uses the **`normalized_features`** method from Part A to obtain their normalized feature values. It must then calculate the Euclidean distance between these two normalized feature dictionaries, using the formula provided in the Appendix. A smaller distance means that the tracks are more similar. If either provided `track_id` does not exist, the method must raise a `ValueError`. The exact wording of the `ValueError` message is not assessed.

  Parameters: Two `track_id` values to compare.

  Return: The Euclidean distance as a float.

- **recommend**

  Returns the **k** tracks that are most similar to the track with the given `track_id`, where **k** is a non-negative integer specifying the number of recommendations to return. The chosen track itself is excluded from the results. The recommendations are returned as a list of (`track_id, distance`) **tuples**, sorted by distance in ascending order, so that the most similar track comes first, with each distance rounded to four decimal places using Python's built-in `round()` function. This method must reuse the `distance` method. If the library contains fewer than **k** tracks besides the given one, all of those tracks are returned. If no track with the given `track_id` exists, or if **k** is negative, the method must raise a `ValueError`. These checks are performed first, so an invalid `track_id` raises a `ValueError` even when **k** is 0. The exact wording of the `ValueError` message is not assessed. For a valid `track_id` with **k** equal to 0, the method returns an empty list.

  Parameters: The query `track_id` and the number of recommendations **k**.

  Return: List of (`track_id, distance`) tuples, sorted by ascending distance.

Note: Sort by the unrounded distance. If two tracks have an equal distance, order them by `track_id` in ascending order. No randomness should be introduced, so that the result is fully reproducible.

Example: `recommend("4yNk9iz9WVJikRFle3XEvn", 2)` might return:

```
[('6NU6jfLqEM6p7X0ZrGCe9D', 0.0413), ('0f7IGNawBHrxVsV0uLAWal', 0.0567)]
```

The identifiers and distances above are illustrative only and do not correspond to the dataset you are given.

Testing: A sample test file, test_music_analyzer.py, is provided as a starting point in the tests folder. You must extend it into a comprehensive test suite without changing the location or name of the file. Your **pytests** should check whether each required method behaves correctly for normal cases, edge cases, and invalid input where relevant.

Task 4: Playlist Generation and Analysis (25 Points)

In this task, you will use the methods implemented in previous tasks to automatically generate a playlist and then describe it with summary statistics. A playlist is built starting from a chosen seed track. **Each following track is the one most similar to the track added before it**, chosen from the tracks not yet used, so the playlist forms a chain that moves gradually away from the seed. This makes it different from `recommend` in Task 3, which compares every track with the seed itself. You will implement both the generator and the analysis methods in a new `Playlist` class, defined in the provided playlist.py file.

Part A: Generate the Playlist (9 Points)

Required methods:

- **__init__**

  Initializes a `Playlist` with an ordered list of `Track` objects, stored in an attribute named `tracks`. The first track in the list is the seed track, and each following track is the one added after it.

  Parameters: List of `Track` objects in the order they appear in the playlist (the seed track first, followed by each track that was added).

  Return: None.

- **generate_playlist**

  Builds and returns a `Playlist` object by starting from the seed track and repeatedly appending the unused track most similar to the one added before it, until the playlist contains the desired number of tracks. Because building a playlist depends on track similarity, the method receives a `MusicAnalyzer` object created in Task 3 and reuses its `distance` method to find the nearest unused track at each step. Because this method creates a new `Playlist` object rather than working on an existing one, it is defined as a static method. As

static methods are not covered in the course, the `@staticmethod` decorator is already provided for you in playlist.py as part of the `Playlist` class, so you only need to implement the body of the method. You can read more about static methods in the Python documentation. If `seed_track_id` does not match any track in the library, or if `length` is less than 1, the method must raise a `ValueError`. The exact wording of the `ValueError` message is not assessed. If `length` is greater than the number of available tracks, the playlist should include all available tracks.

Parameters: A `MusicAnalyzer` object used to measure the distance (similarity) between tracks, the `seed_track_id` of the first track, and `length` (total number of tracks in the playlist, including the seed track).

Return: `Playlist` object containing the selected tracks in playlist order (the seed first, followed by each track added after it).

Note: No randomness should be introduced. At each step, choose the unused track with the smallest distance to the last track added to the playlist. If several tracks are equally close, choose the one whose `track_id` comes first in ascending order.

Part B: Analyze the Playlist (16 Points)

The `Playlist` class represents an ordered playlist and provides the following methods. The helper method `_std_dev`, which computes the sample standard deviation, is already implemented and provided for you, and you should reuse it where appropriate.

Required methods:

- **total_duration**

  Returns the total duration of the playlist in milliseconds, computed as the sum of the `duration_ms` of all its tracks.

  Parameters: None.

  Return: Total duration in milliseconds as an integer.

  Example: If a playlist contains three tracks with durations `180000`, `210000`, and `150000` milliseconds, the method returns `540000`.

- **feature_averages**

  Returns the mean of each of the six audio features across the tracks in the playlist, as a dictionary mapping each feature name to its average, rounded to three decimal places. The method must reuse `audio_features`.

  Parameters: None.

  Return: Dictionary mapping each audio-feature name to its mean value as a float.

  Example: If the `danceability` values of two tracks are `0.515` and `0.642`, their mean is `0.5785`, so the `danceability` entry in the returned dictionary should be `0.579` after

rounding to three decimal places. The same calculation is applied to the other five audio features.

- **popularity_statistics**

  Returns the mean, median, and sample standard deviation of the `popularity` scores of the tracks in the playlist. The `_std_dev` helper method is already provided in playlist.py, but you must implement the `_median` helper method yourself.

  Parameters: None.

  Return: Dictionary with keys "mean", "median", and "std_dev", each mapping to a float.

  Example: For popularity scores `[40, 50, 60, 70]`, the method returns: `{"mean": 55.0, "median": 55.0, "std_dev": 12.9}`.
  For popularity scores `[10, 20, 30]`, the method returns:
  `{"mean": 20.0, "median": 20.0, "std_dev": 10.0}`.

  The mean and standard deviation are rounded to one decimal place using Python's built-in `round()` function. For an even number of values, the median is the average of the two middle values, rounded to one decimal place. For an odd number of values, the middle value is returned unrounded.

- **dominant_mood**

  Returns the mood that occurs most frequently among the playlist's tracks (mode of `mood`).

  Parameters: None.

  Return: Text (one of `Sad, Neutral, Happy`), or None if the playlist is empty. If two moods occur equally often, return the one that appears first in playlist order, so that the result is reproducible. No randomness should be introduced.

  Example: For the moods [`Happy, Sad, Happy, Sad`], the method returns `Happy` because `Happy` and `Sad` occur equally often, but `Happy` appears first in the playlist.

- **artist_variety**

  Returns the number of distinct individual artists across the whole playlist. The method must reuse `artist_names`, so that each artist in a collaboration is counted separately, and an artist appearing on several tracks is counted only once. This measures how much variety the playlist has.

  Parameters: None.

  Return: Integer count of distinct artists.

  Example: Suppose a playlist contains three tracks with the `artists` fields `"Mike Posner;Gigamesh"`, `"Mike Posner"`, and `"Adele"`. The distinct individual artists are Mike Posner, Gigamesh, and Adele, so the method returns 3. Artist names may contain spaces, while only semicolons separate multiple artists.

Note: A playlist created by `generate_playlist` always contains at least one track, because `length` must be at least 1. A `Playlist` object can still be constructed directly with an empty list, and in that case the methods must work without raising an error and must return the following:

- `total_duration` and `artist_variety` return 0.
- `feature_averages` and `popularity_statistics` return an empty dictionary.
- `dominant_mood` returns `None`.

Testing: A sample test file, test_playlist.py, is provided as a starting point in the tests folder. You must extend it into a comprehensive test suite without changing the location or name of the file. Your **pytests** should check whether each required method behaves correctly for normal cases, edge cases, and invalid input where relevant.

Task 5: Interactive Visualization with Dash (Bonus: 10 Points)

In this bonus task, you will build an interactive dashboard using Dash, a Python framework for building analytical web applications. From a chosen seed track, the dashboard presents the recommendations, the playlist, and a summary of that playlist. **It must reuse the classes and methods you implemented in Tasks 2 to 4**. You may use third-party Python libraries, such as **pandas** or **numpy** (this is optional and depends on your approach and preferences). However, to calculate the statistical information displayed in the dashboard, you **must use** the methods you implemented in Tasks 2 to 4. Implement the dashboard in the provided dashboard.py file.

Note: Dash is **not covered in the course**, and completing this task requires **independent research**. You may consult the official Dash documentation and other online tutorials.

Part A: Recommendation Panel (5 Points)

What you should implement:

- A way for the user to select a seed track, such as a dropdown listing the track names or a text field for entering a `track_id`.
- A way for the user to choose the number of recommendations **k**, such as a slider or a number field. Only positive integers may be chosen, so the interface must not allow a value of 0 or below.
- A table showing the recommendations returned by `recommend` for the selected seed track, with columns such as track name, artist or artists, genre, and similarity distance.
- The table must update automatically whenever the seed track or **k** changes.

Figure 1 shows an example of a recommendation panel. It also displays the title, artists, genre, mood, and popularity of the selected seed track, which is optional.

Part B: Generated-Playlist (5 Points)

What you should implement:

- A way for the user to choose the playlist length, such as a slider or a number field.
- A table listing the tracks of the generated playlist in order, with columns such as track name, artist or artists, genre, mood, and duration.
- A summary panel for the playlist generated from the selected seed track with the `generate_playlist` method. The summary must include the number of tracks in the playlist, the playlist's total duration (`total_duration`), its dominant mood (`dominant_mood`), and its artist variety (`artist_variety`).
- Because `total_duration` returns milliseconds, the dashboard must display it in minutes and seconds (for example, `1718000 milliseconds becomes 28 min 38 s`).
- A chart of the playlist's average audio features (`feature_averages`), rescaled with min-max normalization using the minimum and maximum values returned by `feature_ranges`, so that all six features are shown on the same scale.
- The summary and the chart must update automatically whenever the seed track or the playlist length changes.

Figure 2 shows an example of a playlist summary panel.

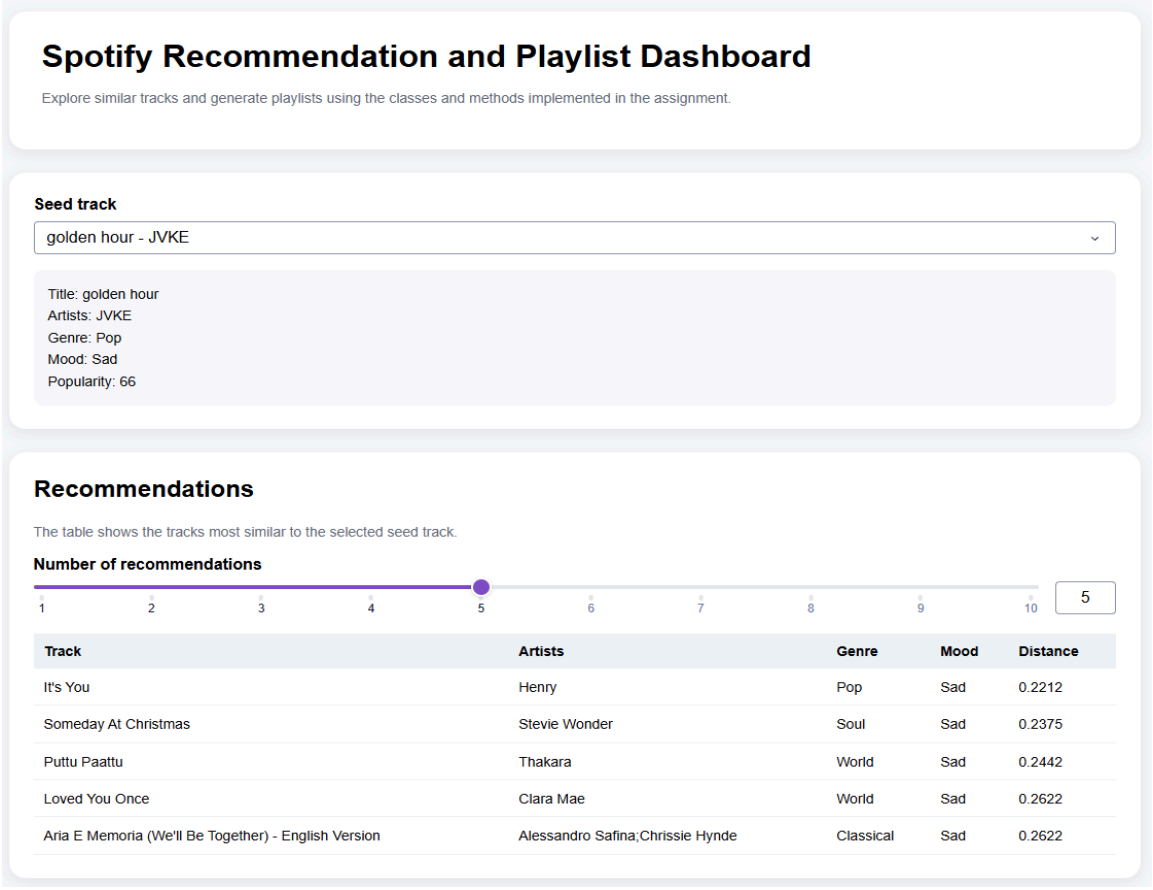


Figure 1: Example of recommendation panel.



Figure 2: Example of generated-playlist.

Note: The user interfaces shown in Figures 1 and 2 are only examples. You are free to design your own interface and layout, provided that your solution implements the required functionality and reuses the classes and methods developed in the previous tasks.

Task 6: Reflection (5 Points)

As you worked through your project, you likely encountered bugs or unexpected behavior that tested your problem-solving skills. In this task, reflect on your debugging experience by describing two meaningful issues you faced. For each one, explain:

1. **What the issue was:** Describe the bug or error clearly.
2. **How you diagnosed it:** Share how you investigated and identified the root cause.
3. **How you resolved it:** Describe the solution you applied and how it fixed the problem.

Please limit your reflection to a maximum of one page (approximately 400-500 words). This length ensures your responses are concise yet detailed enough to demonstrate your technical understanding

and debugging approach. This reflection will be **evaluated subjectively** by the instructors to assess your technical understanding and debugging approach.

Note: This report must also include your **Use of AI statement**, as described on page 2. Include it as a separate section. It does not count toward the one-page limit.

Appendix: Definitions

- **Mean (average):** The sum of all values divided by the number of values.
  Example: for `[2, 4, 5, 7]`, the mean is `(2 + 4 + 5 + 7) / 4 = 4.5`.

- **Median:** The middle value of an ordered list; for an even number of values it is the average of the two middle numbers.
  Example: for `[1, 3, 4]`, the median is 3; for `[1, 2, 3, 4]`, the median is `(2 + 3) / 2 = 2.5`.

- **Mode:** The value that appears most frequently. If several values share the highest frequency, the value that appears earliest in the original list is returned.
  Example: for `[1, 2, 3, 3, 2]`, both 2 and 3 appear twice, but the mode is 2 because 2 appears before 3 in the original list.

- **Standard deviation:** A measure of the spread of data points around the mean. The implementation of the standard deviation method is already provided in the starter code.

- **Min-max normalization:** Rescales a feature to the interval `[0.0, 1.0]` using its minimum m and maximum M across the data: **(x − m) / (M − m)**. If M equals m, the normalized value is defined as 0.0
  Example: if a feature has minimum m = 50 and maximum M = 200, then a value x = 125 becomes `(125 - 50) / (200 - 50) = 0.5`.

- **Euclidean distance:** The straight-line distance between two vectors **u** and **v** of equal length **d**, given by the formula: **dist(u, v) = $\sqrt{((u_1 - v_1)^2 + (u_2 - v_2)^2 + \ldots + (u_d - v_d)^2)}$**
  In words, subtract the vectors component by component, square each difference, add the squares, and take the square root. A smaller distance means the vectors are more similar.
  Example: Each track is represented by a feature vector containing six **normalized** audio features: `danceability`, `energy`, `valence`, `tempo`, `loudness`, and `acousticness`. Therefore, each feature vector has length **d = 6**, with every value rescaled to the interval `[0.0, 1.0]`.
  For two tracks with **normalized** feature vectors:
  u = (0.50, 0.60, 0.20, 0.40, 0.70, 0.65)
  v = (0.50, 0.60, 0.40, 0.40, 0.70, 0.65)
  dist(u, v) = $\sqrt{((0.50 - 0.50)^2 + (0.60 - 0.60)^2 + (0.20 - 0.40)^2 + (0.40 - 0.40)^2 + (0.70 - 0.70)^2 + (0.65 - 0.65)^2)} = \sqrt{0.04}$ = **0.2**.